COMPLETE HTML MASTER NOTES

HTML5 — BASIC → ADVANCED

A full standalone HTML reference for college, projects, interviews, viva and frontend development

Coverage: document structure, every major HTML5 element category, attributes, text, links, lists, tables, forms, validation, media, semantic HTML, accessibility, responsive images, metadata, SEO, security-related HTML practices, iframe/embed, interactive elements, templates, data attributes, Web Components foundations, browser APIs, debugging, best practices and projects.

TABLE OF CONTENTS

1. WEB FUNDAMENTALS AND HTML

1.1 What happens when you open a webpage?

A browser receives resources from a server, parses HTML into a DOM tree, parses CSS into style information, executes JavaScript when present, calculates layout and paints the result on screen. HTML primarily provides structure and meaning.

```
Browser request
      ↓
HTML document
      ↓
HTML parser
      ↓
DOM tree
      ↓
CSS + stylesheets → CSSOM
      ↓
JavaScript can read/change DOM
      ↓
Layout → Paint → Composite
      ↓
Visible webpage
```

1.2 HTML vs CSS vs JavaScript

Technology	Main responsibility	Example
HTML	Structure and meaning	heading, form, table, image
CSS	Presentation and layout	color, spacing, grid, animation
JavaScript	Behavior and logic	click handling, API calls, DOM changes

1.3 What HTML is

HTML means HyperText Markup Language. It uses elements represented by tags to describe document structure. HTML5 is the modern HTML standard used by browsers.

> **Key point:** HTML is not primarily responsible for visual styling or application logic. Keep structure in HTML, presentation in CSS and behavior in JavaScript.

2. HTML DOCUMENT STRUCTURE

2.1 Complete basic document

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">
    <title>My Web Page</title>
</head>
<body>
    <h1>Hello HTML</h1>
    <p>This is my page.</p>
</body>
</html>
```

DOCTYPE

<!DOCTYPE html> declares the document as modern HTML and places browsers into standards mode.

html

The root element contains the head and body. The lang attribute identifies the primary language.

head

Contains metadata and references that normally are not rendered as ordinary page content.

body

Contains the document content presented to the user.

meta charset

```
<meta charset="UTF-8">
```

Viewport

```
<meta name="viewport"
      content="width=device-width, initial-scale=1.0">
```

Title

```
<title>Student Management System</title>
```

Key point: The title is important for browser tabs, bookmarks and search presentation.

2.2 Complete professional starter template

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">

  <meta name="viewport"
        content="width=device-width, initial-scale=1">

  <meta name="description"
        content="A sample HTML5 application.">

  <meta name="author"
        content="Your Name">

  <title>Application Name</title>

  <link rel="icon" href="/favicon.ico">
  <link rel="stylesheet" href="/css/style.css">

  <script src="/js/app.js" defer></script>
</head>

<body>
  <header>
```

```
    <h1>Application</h1>
  </header>

  <main>
    <p>Main content.</p>
  </main>

  <footer>
    <small>&copy; 2026</small>
  </footer>
</body>
</html>
```

3. ELEMENTS, TAGS AND ATTRIBUTES

3.1 Element anatomy

```
<p class="intro" id="welcome">
    Hello World
</p>
```

Part	Meaning
	Opening tag
class="intro"	Attribute name + value
id="welcome"	Unique identifier
Hello World	Text content
	Closing tag

3.2 Nested elements

```
<p>
  This is <strong>important</strong>
  and <em>emphasized</em>.
</p>
```

3.3 Void elements

Void elements do not contain child content and do not use closing tags.

```
<img src="photo.jpg" alt="Photo">
<br>
<hr>
<input type="text">
<meta charset="UTF-8">
<link rel="stylesheet" href="style.css">
```

3.4 Comments

```
<!-- This is an HTML comment -->
```

Key point: Do not put passwords, API keys or sensitive information in comments. Comments remain visible in page source.

4. HEADINGS, PARAGRAPHS AND TEXT

4.1 Headings

```
<h1>Main page heading</h1>
<h2>Major section</h2>
<h3>Subsection</h3>
<h4>Detailed subsection</h4>
<h5>Lower-level heading</h5>
<h6>Lowest heading level</h6>
```

4.2 Paragraphs and line breaks

```
<p>This is a paragraph.</p>
<p>This is another paragraph.</p>

<p>Line one<br>Line two</p>

<hr>
```

> **Key point:** Use CSS margins for visual spacing. Do not create layout using many
 tags.

4.3 Inline semantic text

```
<strong>Important</strong>
<em>Emphasis</em>
<b>Stylistically bold</b>
<i>Alternate voice or term</i>
<mark>Highlighted</mark>
<small>Small print</small>
<del>Deleted</del>
<ins>Inserted</ins>
<s>No longer accurate</s>
<u>Annotation</u>
<sub>2</sub>
<sup>2</sup>
```

4.4 Example

```
<p>
  Water is H<sub>2</sub>O and
  x<sup>2</sup> + y<sup>2</sup> = z<sup>2</sup>.
</p>

<p>
  <mark>Important:</mark>
  Submit the assignment before Friday.
</p>
```

5. QUOTATIONS, CITATIONS AND CODE TEXT

5.1 Short quotation

```
<p>He said, <q>HTML describes structure.</q></p>
```

5.2 Block quotation

```
<blockquote cite="https://example.com">
  A longer quoted passage can be placed here.
</blockquote>
```

5.3 Citation

```
<p>
  Read <cite>HTML Living Standard</cite>
  for the formal specification.
</p>
```

5.4 Code

```
<p>Use the <code>git status</code> command.</p>

<pre>
Line 1
Line 2
Line 3
</pre>

<pre><code>
const x = 10;
console.log(x);
</code></pre>
```

5.5 Keyboard input

```
<p>Press <kbd>Ctrl</kbd> + <kbd>S</kbd> to save.</p>
```

5.6 Variables and output

```
<p>The value is <var>x</var>.</p>
<p>Output: <samp>Hello World</samp></p>
```

6. LINKS AND NAVIGATION

6.1 Basic links

```
<a href="about.html">About</a>

<a href="/contact">Contact</a>

<a href="https://example.com">
  External website
</a>
```

6.2 Target

```
<a href="https://example.com"
   target="_blank"
   rel="noopener noreferrer">
   Open external site
</a>
```

6.3 Page fragments

```
<a href="#contact">Go to contact</a>

<section id="contact">
  <h2>Contact</h2>
</section>
```

6.4 Email and phone

```
<a href="mailto:hello@example.com">
  Email us
</a>

<a href="tel:+911234567890">
  Call us
</a>
```

6.5 Download

```
<a href="files/resume.pdf" download>
  Download Resume
</a>
```

6.6 Navigation

```
<nav aria-label="Primary">
  <a href="/">Home</a>
  <a href="/about">About</a>
  <a href="/projects">Projects</a>
  <a href="/contact">Contact</a>
</nav>
```

Key point: Use meaningful link text. Avoid vague links such as 'click here' when the destination can be described.

7. LISTS

7.1 Unordered list

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>
```

7.2 Ordered list

```
<ol>
  <li>Install tools</li>
  <li>Create project</li>
  <li>Write code</li>
  <li>Test</li>
</ol>
```

7.3 Ordered list controls

```
<ol start="5">
  <li>Fifth item</li>
  <li>Sixth item</li>
</ol>

<ol reversed>
  <li>Three</li>
  <li>Two</li>
  <li>One</li>
</ol>
```

7.4 Description list

```
<dl>
  <dt>HTML</dt>
  <dd>Structures web content.</dd>

  <dt>CSS</dt>
  <dd>Styles web content.</dd>
</dl>
```

7.5 Nested list

```
<ul>
  <li>Frontend
    <ul>
      <li>HTML</li>
      <li>CSS</li>
      <li>JavaScript</li>
    </ul>
  </li>
  <li>Backend</li>
</ul>
```

8. IMAGES AND RESPONSIVE IMAGES

8.1 Basic image

```
<img src="images/logo.png"
     alt="Company logo"
     width="300"
     height="100">
```

8.2 Decorative image

```
<img src="decoration.svg" alt="">
```

An empty alt is appropriate when the image is purely decorative and should be ignored by assistive technology.

8.3 Figure

```
<figure>
  <img src="architecture.png"
       alt="Three layer application architecture">
  <figcaption>
    Three-layer application architecture.
  </figcaption>
</figure>
```

8.4 Responsive image with srcset

```
<img
  src="photo-800.jpg"
  srcset="
    photo-400.jpg 400w,
    photo-800.jpg 800w,
    photo-1200.jpg 1200w"
  sizes="
    (max-width: 600px) 100vw,
    (max-width: 1000px) 80vw,
    800px"
  alt="Mountain landscape">
```

8.5 Picture element

```
<picture>
  <source media="(max-width: 600px)"
          srcset="portrait-mobile.jpg">
  <source media="(min-width: 601px)"
          srcset="portrait-desktop.jpg">
  <img src="portrait-desktop.jpg"
       alt="Portrait">
</picture>
```

8.6 Lazy loading

```
<img src="large.jpg"
     alt="Large photograph"
     loading="lazy"
     width="1200"
     height="800">
```

Key point: Explicit dimensions can reduce layout shift because the browser can reserve space before the image loads.

9. AUDIO, VIDEO AND TRACK

9.1 Audio

```
<audio controls>
  <source src="audio.mp3" type="audio/mpeg">
  <source src="audio.ogg" type="audio/ogg">
  Your browser does not support audio.
</audio>
```

9.2 Video

```
<video controls
       width="800"
       poster="poster.jpg">
  <source src="video.mp4" type="video/mp4">
  Your browser does not support video.
</video>
```

9.3 Captions

```
<video controls>
  <source src="lecture.mp4" type="video/mp4">
  <track
    kind="captions"
    src="captions-en.vtt"
    srclang="en"
    label="English"
    default>
</video>
```

9.4 Common media attributes

Attribute	Purpose
controls	Displays browser controls
autoplay	Requests automatic playback; often restricted
muted	Starts without audio
loop	Repeats playback
poster	Video placeholder image
preload	Hints how media should be loaded
playsinline	Supports inline playback on compatible mobile browsers

10. TABLES — COMPLETE

10.1 Basic table

```
<table>
  <tr>
    <th>Name</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>Aryan</td>
    <td>21</td>
  </tr>
</table>
```

10.2 Professional accessible table

```
<table>
  <caption>Student Marks</caption>

  <thead>
    <tr>
      <th scope="col">Student</th>
      <th scope="col">HTML</th>
      <th scope="col">CSS</th>
      <th scope="col">JS</th>
    </tr>
  </thead>

  <tbody>
    <tr>
      <th scope="row">Aryan</th>
      <td>92</td>
      <td>88</td>
      <td>85</td>
    </tr>
  </tbody>

  <tfoot>
    <tr>
      <th scope="row">Average</th>
      <td>92</td>
      <td>88</td>
      <td>85</td>
    </tr>
  </tfoot>
</table>
```

10.3 colspan and rowspan

```
<table>
  <tr>
    <th colspan="2">Student</th>
  </tr>
  <tr>
    <td rowspan="2">Aryan</td>
    <td>HTML</td>
  </tr>
  <tr>
    <td>CSS</td>
  </tr>
</table>
```

10.4 Column groups

```
<table>
  <colgroup>
    <col>
    <col span="2" class="marks">
  </colgroup>
  ...
</table>
```

Key point: Never use tables for general page layout. Tables are intended for relationships between rows and columns of data.

11. FORMS — COMPLETE

11.1 Form anatomy

```
<form action="/register"
      method="post"
      autocomplete="on">

  <label for="username">Username</label>
  <input id="username"
         name="username"
         type="text"
         required>

  <button type="submit">Register</button>
</form>
```

11.2 Text controls

```
<input type="text">
<input type="search">
<input type="email">
<input type="url">
<input type="tel">
<input type="password">
<textarea name="message" rows="5"></textarea>
```

11.3 Numeric/date controls

```
<input type="number" min="0" max="100" step="1">
<input type="range" min="0" max="100" value="50">
<input type="date">
<input type="time">
<input type="datetime-local">
<input type="month">
<input type="week">
```

11.4 Selection controls

```
<label>
  <input type="checkbox" name="skills" value="html">
  HTML
</label>

<label>
  <input type="radio" name="gender" value="male">
  Male
</label>

<label>
  <input type="radio" name="gender" value="female">
  Female
</label>
```

11.5 Select

```
<label for="country">Country</label>
<select id="country" name="country">
  <option value="">Choose...</option>
  <option value="in">India</option>
  <option value="us">United States</option>
</select>
```

11.6 Multiple select

```
<select name="skills" multiple size="4">
  <option value="html">HTML</option>
  <option value="css">CSS</option>
  <option value="js">JavaScript</option>
  <option value="react">React</option>
</select>
```

11.7 Datalist

```
<label for="browser">Browser</label>
<input id="browser" name="browser" list="browsers">

<datalist id="browsers">
  <option value="Chrome">
  <option value="Firefox">
  <option value="Edge">
</datalist>
```

12. NATIVE FORM VALIDATION

12.1 Required

```
<input type="text" name="name" required>
```

12.2 Length

```
<input type="text"
       minlength="3"
       maxlength="30">
```

12.3 Numeric constraints

```
<input type="number"
       min="18"
       max="100"
       step="1">
```

12.4 Pattern

```
<input
  name="username"
  pattern="[A-Za-z0-9_]{4,16}"
  title="4–16 letters, numbers or underscores"
  required>
```

12.5 Email

```
<input type="email"
       name="email"
       required>
```

12.6 Complete form

```
<form action="/register" method="post">
  <fieldset>
    <legend>Create Account</legend>

    <label for="name">Full name</label>
    <input id="name"
           name="name"
           type="text"
           minlength="2"
           required>

    <label for="email">Email</label>
    <input id="email"
           name="email"
           type="email"
           required>

    <label for="password">Password</label>
    <input id="password"
           name="password"
           type="password"
           minlength="8"
           required>

    <button type="submit">Create account</button>
  </fieldset>
</form>
```

Key point: Client-side validation improves user experience but is not a security boundary. A server must validate and authorize submitted data independently.

13. SEMANTIC HTML5

Semantic elements describe the role of content. They make documents easier to understand for developers, search systems and assistive technologies.

```
<body>
  <header>
    <a href="/">Brand</a>
  </header>

  <nav aria-label="Primary navigation">
    ...
  </nav>

  <main>
    <article>
      <header>
        <h1>Article title</h1>
      </header>

      <section>
        <h2>Introduction</h2>
        <p>...</p>
      </section>

      <section>
        <h2>Main topic</h2>
        <p>...</p>
      </section>

      <footer>Article metadata</footer>
    </article>

    <aside>
      Related content
    </aside>
  </main>

  <footer>
    Copyright
  </footer>
</body>
```

When to use what?

Element	Typical meaning
header	Introductory/navigation content for a page or section
nav	Major navigation links
main	Primary unique content of the page
section	Thematic grouping, usually with a heading
article	Self-contained distributable content
aside	Tangential/related content
footer	Footer information for page or section
div	Generic block when no semantic element fits
span	Generic inline container when no semantic element fits

14. GLOBAL ATTRIBUTES

Global attributes can generally be used on HTML elements, subject to the individual element's rules.

Attribute	Purpose / example
id	Unique element identifier
class	Reusable classification for CSS/JS
title	Advisory tooltip/text
lang	Language of content
dir	Text direction: ltr, rtl, auto
hidden	Marks content as not currently relevant/rendered
tabindex	Controls focusability/order; use carefully
data-*	Custom data attributes
contenteditable	Allows editing of content
draggable	Controls drag behavior hint
spellcheck	Spell-checking hint
translate	Translation hint
role	ARIA semantic role when needed
aria-*	Accessibility state/property
style	Inline CSS; use sparingly

14.1 id and class

```
<section id="projects" class="section featured">
  ...
</section>
```

14.2 data-*

```
<button
  class="delete-btn"
  data-user-id="42"
  data-role="student">
  Delete
</button>
```

14.3 tabindex

```
<button tabindex="0">Focusable</button>
```

Key point: Do not use positive tabindex values as a shortcut for designing keyboard navigation. Native interactive controls should normally be used.

15. HTML ENTITIES

Entities represent reserved characters or characters that are difficult to type directly.

```
&lt;    <
&gt;    >
&amp;   &
&quot;  "
&apos;  '
   non-breaking space
&copy; ©
&reg;   ®
&trade; ™
&euro;  €
&times; ×
&check; ✓
```

Example

```
<p>Use &lt;div&gt; when displaying literal HTML.</p>
<p>&copy; 2026 My Company</p>
```

16. METADATA AND THE

```
<head>
  <meta charset="UTF-8">

  <meta name="viewport"
        content="width=device-width, initial-scale=1">

  <meta name="description"
        content="Page description">

  <meta name="robots"
        content="index,follow">

  <link rel="canonical"
        href="https://example.com/page">

  <link rel="icon"
        href="/favicon.ico">

  <link rel="stylesheet"
        href="/css/style.css">

  <script src="/js/app.js" defer></script>

  <title>Page Title</title>
</head>
```

16.1 Canonical

A canonical link can indicate the preferred URL for duplicate or near-duplicate pages.

```
<link rel="canonical"
      href="https://example.com/products">
```

17. SEO-ORIENTED HTML

17.1 Strong page structure

```
<title>HTML Master Notes — Complete Guide</title>

<meta name="description"
      content="A complete HTML guide from beginner to advanced.">

<h1>Complete HTML Guide</h1>

<h2>HTML Forms</h2>
<h2>HTML Tables</h2>
<h2>HTML Accessibility</h2>
```

17.2 Good link text

```
<!-- Better -->
<a href="/html-forms">Learn HTML forms</a>

<!-- Weak -->
<a href="/html-forms">Click here</a>
```

17.3 Image alt

```
<img src="dashboard.png"
     alt="Student dashboard showing course progress">
```

17.4 Structured content

Use semantic sections, useful headings, descriptive titles, meaningful URLs and accessible navigation. SEO is not achieved by stuffing keywords into HTML.

18. ACCESSIBILITY AND ARIA

18.1 First rule: use native HTML

A real button should be a button, not a div pretending to be one. Native controls already provide keyboard and accessibility behavior that custom controls must otherwise recreate.

```
<!-- Correct -->
<button type="button">Open menu</button>

<!-- Avoid replacing it with -->
<div role="button">Open menu</div>
```

18.2 Labels

```
<label for="email">Email address</label>
<input id="email" type="email" name="email">
```

18.3 Landmark navigation

```
<nav aria-label="Primary navigation">
  ...
</nav>

<nav aria-label="Footer navigation">
  ...
</nav>
```

18.4 ARIA state

```
<button
  aria-expanded="false"
  aria-controls="main-menu">
  Menu
</button>

<nav id="main-menu">
  ...
</nav>
```

18.5 Images

```
<!-- Informative -->
<img src="chart.png"
     alt="Revenue rose from 10 lakh to 15 lakh">

<!-- Decorative -->
<img src="divider.svg" alt="">
```

Key point: ARIA supplements semantic HTML; it does not make poor HTML automatically accessible.

19. IFRAMES AND EMBEDDED CONTENT

19.1 iframe

```
<iframe
  src="https://example.com"
  title="Embedded example"
  width="800"
  height="500">
</iframe>
```

19.2 Sandbox

```
<iframe
  src="/embedded.html"
  title="Embedded content"
  sandbox
  width="800"
  height="500">
</iframe>
```

19.3 Embed and object

```
<embed
  src="document.pdf"
  type="application/pdf"
  width="800"
  height="600">

<object
  data="document.pdf"
  type="application/pdf"
  width="800"
  height="600">
  <p>PDF cannot be displayed.</p>
</object>
```

Key point: Embedded third-party content can introduce performance, privacy and security considerations. Use the least privilege and permissions necessary.

20. INTERACTIVE HTML

20.1 Details and summary

```
<details>
  <summary>What is HTML?</summary>
  <p>HTML structures web documents.</p>
</details>
```

20.2 Dialog

```
<dialog id="confirmDialog">
  <p>Are you sure?</p>
  <form method="dialog">
    <button value="cancel">Cancel</button>
    <button value="ok">OK</button>
  </form>
</dialog>
```

20.3 Progress

```
<label for="progress">Upload progress</label>
<progress id="progress"
          value="70"
          max="100">
  70%
</progress>
```

20.4 Meter

```
<label for="score">Score</label>
<meter id="score"
       min="0"
       max="100"
       value="82">
  82%
</meter>
```

20.5 Output

```
<form oninput="result.value =
  Number(a.value) + Number(b.value)">
  <input id="a" type="number" value="10">
  +
  <input id="b" type="number" value="20">
  =
  <output name="result">30</output>
</form>
```

21. TEMPLATE, SLOT AND WEB COMPONENT FOUNDATIONS

21.1 template

The template element stores inert markup that can later be cloned with JavaScript.

```
<template id="card-template">
  <article class="card">
    <h2 class="title"></h2>
    <p class="description"></p>
  </article>
</template>
```

21.2 slot

```
<template id="my-card">
  <article>
    <slot name="title">Default title</slot>
    <slot>Default content</slot>
  </article>
</template>
```

21.3 Custom element concept

```
class UserCard extends HTMLElement {
  connectedCallback() {
    this.innerHTML = `
      <article>
        <h2>${this.getAttribute("name")}</h2>
      </article>
    `;
  }
}

customElements.define("user-card", UserCard);

<user-card name="Aryan"></user-card>
```

Key point: Web Components combine custom elements, templates and Shadow DOM concepts. They become especially useful when building reusable platform-level components.

22. DATA ATTRIBUTES

```
<button
  class="product"
  data-product-id="101"
  data-price="499">
  Add to cart
</button>
```

Custom data attributes are useful for associating non-visible application data with an element. JavaScript can access them through element.dataset.

```
const button = document.querySelector(".product");

console.log(button.dataset.productId);
console.log(button.dataset.price);
```

23. MICRODATA / STRUCTURED DATA BASICS

Microdata provides a way to annotate content with machine-readable vocabulary. JSON-LD is often preferred for structured data because it keeps metadata separate from visible markup.

```
<div itemscope itemtype="https://schema.org/Person">
  <span itemprop="name">Aryan Gupta</span>
  <span itemprop="jobTitle">Developer</span>
</div>
```

For production SEO implementations, use structured-data guidance appropriate to the content type and validate the resulting markup.

24. HTML SECURITY CONSIDERATIONS

24.1 Never trust submitted HTML

Client-side markup and validation can be modified by users. Server-side validation, output encoding and appropriate security headers are required in real applications.

24.2 External links

```
<a href="https://external.example"
   target="_blank"
   rel="noopener noreferrer">
   External
</a>
```

24.3 Iframes

```
<iframe
  src="/embedded"
  sandbox="allow-scripts"
  title="Embedded application">
</iframe>
```

24.4 Forms

Never treat required, pattern, maxlength or other browser validation as a security boundary. Validate all important constraints on the server.

25. PERFORMANCE-ORIENTED HTML

```
<img
  src="photo.jpg"
  alt="Product photo"
  width="800"
  height="600"
  loading="lazy"
  decoding="async">
```

Use responsive images, reserve dimensions, avoid unnecessary embeds, load scripts appropriately and keep the initial document useful even when enhancement scripts have not executed.

26. PROGRESSIVE ENHANCEMENT

Start with functional HTML. Add CSS for presentation. Add JavaScript for enhanced interactions. A good page should remain understandable and usable when possible without client-side enhancement.

```
<!-- Base functionality -->
<form action="/search" method="get">
  <label for="q">Search</label>
  <input id="q" name="q">
  <button type="submit">Search</button>
</form>

<!-- JavaScript can later add instant suggestions,
     without removing the basic form behavior. -->
```

27. HTML + CSS + JS INTEGRATION

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport"
        content="width=device-width,initial-scale=1">
  <title>Interactive Card</title>
  <link rel="stylesheet" href="style.css">
  <script src="app.js" defer></script>
</head>
<body>

  <main>
    <button id="themeButton"
            type="button">
      Toggle theme
    </button>

    <section class="card" id="card">
      <h1>Full Stack</h1>
      <p>HTML + CSS + JS</p>
    </section>
  </main>

</body>
</html>
```

The HTML creates the structure; CSS can style .card; JavaScript can select #themeButton and change a class or attribute. This separation is the foundation for frontend frameworks such as React.

28. DOM RELATIONSHIP AND BROWSER PARSING

28.1 DOM tree

```
document
■■■ html
    ■■■ head
    ■   ■■■ meta
    ■   ■■■ title
    ■■■ body
        ■■■ header
        ■■■ main
            ■■■ h1
            ■■■ p
```

28.2 Parent, child and sibling

```
<main>
  <h1>Title</h1>
  <p>Text</p>
</main>
```

• main is the parent of h1 and p.

• h1 and p are siblings.

• main is an ancestor of h1.

• h1 and p are descendants of main.

28.3 Why DOM knowledge matters

JavaScript interacts with HTML primarily through the DOM. React and other frontend libraries ultimately cause updates to the browser's rendered document through their own rendering systems.

29. VALIDATION AND DEBUGGING

29.1 Browser DevTools

• Elements: inspect the actual DOM.

• Styles/Computed: determine why CSS appears or does not appear.

• Console: inspect JavaScript errors and messages.

• Network: find missing HTML/CSS/JS/image resources and HTTP status codes.

• Accessibility tools: inspect names, roles and keyboard behavior.

29.2 Common errors

Symptom	Check
Image not shown	src path, filename case, Network status
CSS not loaded	link href, file path, Network
JS runs too early	Use defer or place script appropriately
Form value missing	name attribute
Keyboard cannot reach control	Use native button/link/input
Heading hierarchy confusing	Check h1-h6 structure
Mobile layout broken	Viewport meta + responsive CSS
External iframe fails	URL, embedding policy, CSP/security restrictions

29.3 Validation mindset

When something breaks, isolate the smallest failing example. Confirm the HTML structure first, then resources, then CSS, then JavaScript. Avoid changing many unrelated lines at once.

30. COMPLETE COMMON HTML TAG REFERENCE

Element	Category	Use
html	Document	Root
head	Document	Metadata container
body	Document	Visible document content
title	Metadata	Document title
meta	Metadata	Metadata
link	Metadata	External resource/link
style	Metadata	Embedded CSS
script	Programming	JavaScript/resource
noscript	Programming	Fallback when scripting unavailable
h1-h6	Text	Headings
p	Text	Paragraph
br	Text	Line break
hr	Text	Thematic break
strong/em	Text	Importance/emphasis
b/i/u/s	Text	Presentational/semantic nuances
mark/small/del/ins	Text	Marked/small/deleted/inserted
sub/sup	Text	Subscript/superscript
abbr	Text	Abbreviation
time	Text	Date/time
data	Text	Machine-readable value
q	Quotation	Inline quote
blockquote	Quotation	Block quote
cite	Quotation	Citation/title
code/pre	Code	Code/preformatted text
kbd/samp/var	Code	Keyboard/output/variable
a	Links	Hyperlink
nav	Semantic	Navigation
ul/ol/li	Lists	Lists
dl/dt/dd	Lists	Description list
img	Media	Image
picture/source	Media	Responsive/source selection
audio	Media	Audio
video	Media	Video
track	Media	Timed text
figure/figcaption	Media	Figure + caption
table	Data	Table

Element	Category	Use
caption	Data	Table caption
thead/tbody/tfoot	Data	Table sections
tr/th/td	Data	Table cells
colgroup/col	Data	Column definitions
form	Forms	Form container
label	Forms	Control label
input	Forms	Input control
button	Forms	Button
select/option/optgroup	Forms	Choices
textarea	Forms	Multiline text
datalist	Forms	Suggested values
fieldset/legend	Forms	Group controls
output	Forms	Calculated output
progress	Forms	Progress
meter	Forms	Scalar measurement
header	Semantic	Intro/header
main	Semantic	Main content
section	Semantic	Thematic section
article	Semantic	Self-contained content
aside	Semantic	Related/tangential content
footer	Semantic	Footer
div	Generic	Generic block container
span	Generic	Generic inline container
iframe	Embedding	Embedded browsing context
embed/object	Embedding	Embedded external resource
details/summary	Interactive	Disclosure widget
dialog	Interactive	Dialog
template	Advanced	Inert reusable markup
slot	Advanced	Web Component insertion point
canvas	Graphics	Scriptable bitmap

31. COMPLETE FORM INPUT REFERENCE

type	Typical purpose
text	General text
search	Search field
email	Email
url	URL
tel	Telephone
password	Password
number	Numeric input
range	Range slider
date	Calendar date
time	Time
datetime-local	Local date and time
month	Month
week	Week
checkbox	Independent/multiple choices
radio	Single choice from group
file	File selection
color	Color selection
hidden	Hidden submitted value
submit	Submit form
reset	Reset form
button	Generic button
image	Image submit control

31.1 File input

```
<form method="post"
      enctype="multipart/form-data">
  <label for="resume">Resume</label>
  <input id="resume"
         name="resume"
         type="file"
         accept=".pdf,.doc,.docx">
  <button type="submit">Upload</button>
</form>
```

31.2 Multiple files

```
<input type="file"
       name="photos"
       accept="image/*"
       multiple>
```

32. COMPLETE HTML PROJECT — STUDENT PORTFOLIO

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
        content="width=device-width,initial-scale=1">
  <meta name="description"
        content="Student developer portfolio">
  <title>Student Developer Portfolio</title>
</head>

<body>
  <header>
    <h1>Aryan Gupta</h1>
    <p>Computer Science Student & Developer</p>
  </header>

  <nav aria-label="Primary">
    <a href="#about">About</a>
    <a href="#skills">Skills</a>
    <a href="#projects">Projects</a>
    <a href="#contact">Contact</a>
  </nav>

  <main>
    <section id="about">
      <h2>About Me</h2>
      <p>I build web applications and learn full stack development.</p>
    </section>

    <section id="skills">
      <h2>Skills</h2>
      <ul>
        <li>HTML</li>
        <li>CSS</li>
        <li>JavaScript</li>
        <li>React</li>
        <li>SQL</li>
        <li>Spring Boot</li>
      </ul>
    </section>

    <section id="projects">
      <h2>Projects</h2>

      <article>
        <h3>Student Management System</h3>
        <p>Web application for managing student records.</p>
        <a href="/projects/student-management">
          View project
        </a>
      </article>

      <article>
        <h3>Task Manager</h3>
        <p>Application for creating and tracking tasks.</p>
        <a href="/projects/task-manager">
          View project
        </a>
      </article>
    </section>

    <section id="contact">
      <h2>Contact</h2>

      <form action="/contact" method="post">
        <div>
          <label for="name">Name</label>
          <input id="name"
                 name="name"
                 type="text"
                 required>
        </div>

        <div>
```

```
          <label for="email">Email</label>
          <input id="email"
                 name="email"
                 type="email"
                 required>
        </div>

        <div>
          <label for="message">Message</label>
          <textarea id="message"
                    name="message"
                    rows="6"
                    required></textarea>
        </div>

        <button type="submit">Send message</button>
      </form>
    </section>
  </main>

  <footer>
    <small>&copy; 2026 Aryan Gupta</small>
  </footer>
</body>
</html>
```

Project preview: a complete semantic portfolio page with accessible navigation, sections, project articles and a contact form. CSS can later turn this semantic structure into a professional responsive design.

33. REGISTRATION FORM PROJECT

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
        content="width=device-width,initial-scale=1">
  <title>Registration</title>
</head>
<body>

<main>
  <h1>Create Account</h1>

  <form action="/register"
        method="post"
        autocomplete="on">

    <fieldset>
      <legend>Personal Information</legend>

      <p>
        <label for="fullName">Full name</label>
        <input id="fullName"
               name="fullName"
               type="text"
               autocomplete="name"
               required
               minlength="2">
      </p>

      <p>
        <label for="email">Email</label>
        <input id="email"
               name="email"
               type="email"
               autocomplete="email"
               required>
      </p>

      <p>
        <label for="phone">Phone</label>
        <input id="phone"
               name="phone"
               type="tel"
               autocomplete="tel">
      </p>
    </fieldset>

    <fieldset>
      <legend>Account</legend>

      <p>
        <label for="password">Password</label>
        <input id="password"
               name="password"
               type="password"
               autocomplete="new-password"
               minlength="8"
               required>
      </p>

      <p>
        <label for="country">Country</label>
        <select id="country"
                name="country"
                required>
          <option value="">Select</option>
          <option value="in">India</option>
          <option value="us">United States</option>
        </select>
      </p>
    </fieldset>

    <p>
      <label>
```

```
        <input type="checkbox"
               name="terms"
               value="accepted"
               required>
        I accept the terms.
      </label>
    </p>

    <button type="submit">Register</button>
  </form>
</main>

</body>
</html>
```

34. INTERVIEW QUESTIONS — HTML

• What is HTML?

• What is HTML5?

• What is the purpose of <!DOCTYPE html>?

• What is the difference between an element and a tag?

• What is an attribute?

• What are void elements?

• Difference between id and class?

• Why should semantic HTML be used?

• Difference between section and div?

• Difference between article and section?

• Why is alt required/important on images?

• What is srcset?

• What is the picture element?

• Difference between strong and b?

• Difference between em and i?

• Difference between ol and ul?

• What are thead, tbody and tfoot?

• What is colspan/rowspan?

• Why should tables not be used for layout?

• What is the purpose of label?

• Why does a form control need a name?

• GET vs POST at the HTTP level?

• What does required do?

• What does pattern do?

• Can HTML validation replace server validation?

• What is semantic HTML?

• What is ARIA?

• When should ARIA be used?

• What is progressive enhancement?

• What is the DOM?

• What is an iframe?

• What does sandbox do?

• What are data-* attributes?

• What is template?

• What are Web Components?

• What is canvas?

• How does defer affect a script?

• Why is viewport meta important?

• What is canonical URL?

• What is lazy loading?

• How do you debug a broken image?

• How do you debug a form that submits no value?

35. VIVA QUESTIONS — SHORT ANSWERS

Question	Answer
HTML full form?	HyperText Markup Language.
CSS full form?	Cascading Style Sheets.
HTML programming language?	No; it is a markup language.
Root HTML element?	html.
Visible content?	Usually inside body.
Metadata?	Information about the document, usually in head.
Unique identifier?	id.
Reusable identifier?	class.
Link element?	a.
Image element?	img.
Form container?	form.
Accessible form label?	label associated with the control.
Semantic main content?	main.
Navigation?	nav.
Self-contained content?	article.
Generic block?	div.
Generic inline?	span.
Responsive image attribute?	srcset, often with sizes.
Native disclosure?	details + summary.
Scriptable bitmap?	canvas.

36. PRACTICE PROBLEMS

• Create an HTML page containing all six heading levels and explain their hierarchy.

• Create a navigation menu with internal page fragments.

• Create a nested unordered list representing a full stack roadmap.

• Create a marks table with caption, thead, tbody, tfoot and scope attributes.

• Create a registration form using at least ten input types.

• Add native validation to email, password, age and username controls.

• Create a responsive image using srcset and sizes.

• Create a picture element with separate mobile and desktop images.

• Create an accessible video with captions.

• Create a semantic blog article page.

• Create a product page using figure, figcaption, time, data and article.

• Create a dialog using dialog and form method=dialog.

• Create a reusable template for product cards.

• Add data-product-id to every product button.

• Debug an intentionally broken HTML file using DevTools.

• Build a complete portfolio without using CSS at first. Verify that the semantic HTML is still understandable.

37. FINAL HTML CHEAT SHEET

Document

```
<!doctype html>
<html lang="en">
<head> ... </head>
<body> ... </body>
</html>
```

Text

```
h1-h6   p   br   hr
strong  em  mark  small
del     ins sub sup
abbr    time data
```

Structure

```
header  nav  main  section
article aside footer
div span
```

Content

```
a img figure figcaption
ul ol li
dl dt dd
table caption thead tbody tfoot tr th td
```

Forms

```
form label input button
select option optgroup
textarea datalist
fieldset legend output
progress meter
```

Media

```
audio video source track
picture iframe embed object
canvas
```

Interactive / Advanced

```
details summary dialog
template slot
data-* attributes
Web Components foundations
```

Remember

• HTML = structure + semantics.

• CSS = presentation.

• JavaScript = behavior.

• Use semantic HTML before ARIA.

• Associate every form control with a label when a visible label is appropriate.

• Use meaningful alt text for informative images.

• Use tables for data, not layout.

• Use native interactive controls.

• Client-side validation is not security.

• Keep HTML valid, readable and logically nested.

• Build mobile-friendly structure from the beginning.

• Learn the DOM because JavaScript and frontend frameworks interact with document structure.

END — COMPLETE HTML MASTER NOTES

Next recommended volume: Complete CSS Master Notes — selectors → cascade → box model → Flexbox → Grid → responsive design → typography → modern CSS functions → animations → architecture → projects.

This volume is intended as the standalone HTML foundation of the larger Full Stack Development series.